

AGENTIC AI

Studybook and Exam Conspect

From language models to reliable autonomous systems

Edition 9 September 2026. Agent architecture, research readings, and assessed engineering practice with Gemini, LangGraph, and the OpenAI Agents SDK.

From language models to reliable autonomous systems.

This studybook develops agent architecture through implementation, evaluation, and operating decisions. The 2026-09-09 edition combines the original 13-chapter structure with dated research readings and assessed labs using Gemini, LangGraph, and the OpenAI Agents SDK. Each chapter states an observable acceptance check.

How to use this studybook

1. Follow the chapter sequence below; complete each assessed practice and self-test.
2. Use repaired Labs 3-4 before the four assessed Labs 5-8; finish with teaching/CAPSTONE.md.
3. Record failures as well as successes. Fixtures test code; live experiments test a configured system.
4. Read research findings with their stated limitations.

Course map

Part	Chapters	Central question
Foundations	1-3	What is an agent, and how does its loop work?
Architecture	4-7	How should control, tools, knowledge, and state be composed?
Coordination	8-10	How should agents collaborate and reason over longer horizons?
Assurance	11-13	How do we measure, secure, operate, and govern the system?

The unifying mental model

An agent is not a prompt with a fashionable name. It is a controlled system:

goal -> context/profile -> reason/plan -> proposed action -> runtime validation -> tool/environment -> observation -> updated state

The loop is bounded by step, token, time, and cost budgets. Evaluation measures both the outcome and the trajectory. Security is enforced at every action and trust boundary. Production engineering versions and observes every moving part.

01. From Language Models to Agents

Agency appears when an LLM is placed inside a bounded loop with tools, memory, and control logic.

Learning objectives

- Define classical and LLM-based agents
- Distinguish chatbots, workflows, and agents
- Explain the perceive-reason-act loop
- Choose the least autonomy that solves a task

Core notes

The composition changed, not the model paradigm

A classifier maps text to a label and a chatbot maps conversation to a reply. An agent maps a goal to a sequence of actions in an environment and chooses those actions as the task unfolds. The practical shift is the composition LLM + tools + loop + memory.

- Instruction following and function calling make outputs executable.
- Explicit reasoning helps choose and order actions.
- Context and retrieval carry task state and external evidence.

Classical agency still supplies the core definition

An agent perceives an environment through sensors and acts through actuators. A rational agent selects actions expected to maximize a performance measure given its percepts and knowledge. LLM agents replace a hand-written or learned policy with in-context natural-language reasoning.

- Properties: autonomy, reactivity, pro-activeness, social ability.
- Hard environments are partially observable, stochastic, dynamic, and sequential.
- LLM agents gain generality but inherit reliability, cost, and evaluation problems.

Four components recur in every implementation

The profile defines identity, constraints, tools, and output protocol. Memory supplies short- and long-term state. Planning turns goals into steps. Action exposes controlled effects through tools. Frameworks mainly differ in how they wire and persist these parts.

- Profile is the behavioral contract.
- Memory is information re-supplied to a stateless model.
- Planning selects the next step or an explicit plan.
- Action is mediated by a validating runtime.

Agency is a loop, not a single forward pass

The agent observes the goal and environment, reasons about the next step, acts through a tool, receives a new observation, and repeats until completion or a budget limit. The loop turns model output into state-changing behavior.

- Observe: user goal, tool results, environment state.
- Reason: choose the next action from current evidence.
- Act: request a tool call through the runtime.
- Stop: final answer, step cap, cost cap, time cap, or human escalation.

Control flow separates chatbots, workflows, and agents

A chatbot produces a reply without actions. A workflow follows code-defined paths that may contain LLM calls and tools. An agent lets the model dynamically direct process and tool use. This is a spectrum rather than a binary label.

- Workflows are more predictable, testable, and economical.
- Agents are more flexible when the correct path cannot be written in advance.
- Moving toward autonomy raises variance, latency, cost, and safety surface.

The engineering rule is minimum sufficient autonomy

Start with a single call or fixed workflow and add model-directed decisions only when the task requires them. Narrow scope, explicit evaluation, and human checkpoints are more reliable than an open-ended agent that can do anything.

- Use execution-grounded tasks where success can be checked.
- Bound long horizons because errors compound.
- Treat the production agent as a stack: model, tools, state, orchestration, eval, and guardrails.

A baseline makes autonomy a testable choice

Choose a small task with observable success before building an agent. Compare a fixed workflow with a model-directed loop on the same inputs. Treat the decision to add autonomy as an engineering hypothesis.

- Write expected outputs before implementation.
- Keep task inputs and resource limits comparable.
- Retain the simpler design when it meets the requirements.

Exam-ready summary

- Agent = LLM reasoning core + tools + loop + memory.
- Profile, memory, planning, and action are the reusable anatomy.
- Autonomy is a design variable; choose the leftmost workable point.
- Reliability comes from the system around the model.

Self-test

1. Give the classical definition of an agent and reframe it for an LLM system.
2. Why is an agent a composition rather than a new model paradigm?
3. Compare chatbot, workflow, and agent by control flow and tool use.
4. Name the four agent components and the responsibility of each.
5. Why are typical LLM-agent environments difficult?
6. How would you test whether a fixed workflow is sufficient for a task?

Assessed practice

Choose a catalog lookup or arithmetic task. Write six cases, including empty input and an unknown item. Define a pass condition and a call limit. Explain what dynamic decision, if any, needs a model.

Acceptance check: Submit the cases and an architecture choice before running a model. Credit follows the evidence, including a decision to keep a fixed workflow.

Lab: labs/05_gemini_bounded_tools.ipynb

Reading and evidence

Use the classroom baseline and its explicit acceptance tests. This activity is a teaching design.

02. The LLM as a Reasoning Engine

Agents inherit the stochastic, token-based nature of LLMs; prompting, structured output, retrieval, and adaptation compensate for different limitations.

Learning objectives

- Explain next-token prediction and sampling
- Use major prompting strategies appropriately
- Distinguish prompting, RAG, and fine-tuning
- Design machine-readable outputs for control loops

Core notes

The engine predicts one token at a time

An autoregressive LLM defines a probability distribution for the next token given previous tokens, samples one, appends it, and repeats. This makes agent behavior stochastic, gives the model no built-in persistent state, and represents plans, actions, and observations as tokens.

- Low temperature favors predictable control-loop behavior.
- Top-p limits sampling to a probability-mass nucleus.
- Input and output tokens both affect latency and cost.

The context window is a shared budget

System instructions, tool definitions, retrieved evidence, memory, conversation, and generated reasoning all compete for a fixed window. Agent loops resend a growing transcript, so relevance and compression matter.

- Long contexts cost more and take longer.
- Evidence in the middle may be underused.
- The cheapest token is the one the system does not send.

The prompt programs behavior in context

In-context learning teaches a task through instructions and examples without changing model weights. A robust prompt separates instruction, context, input, and output indicator; an agent profile also includes tool contracts and the protocol the harness parses.

- Zero-shot is fast but can be brittle on edge cases.
- Few-shot demonstrations stabilize task pattern and format.
- Clear output examples reduce parse failures.

Different reasoning techniques buy different guarantees

Chain-of-thought externalizes intermediate steps. Self-consistency samples several chains and votes.

Program-aided language models generate code and delegate exact computation to an interpreter.

Reasoning models spend additional test-time compute on difficult problems.

- Compare direct answers and reasoning prompts on the target task.
- Test voting against a baseline with a declared call budget.
- Use executable programs when exact computation matters.
- Pay for deliberate reasoning only on hard steps.

Structured output closes the software loop

Structured-output support varies by provider, model, and schema. Native function calling supplies a protocol for tool requests. The application must validate arguments, check policy, and handle invalid or incomplete responses.

- Schema validity is not semantic correctness.
- Tool descriptions steer selection and argument filling.
- The model requests; application code executes.

Prompting, retrieval, and fine-tuning solve different problems

Prompting changes behavior quickly. RAG injects current or private knowledge at query time. Fine-tuning changes model behavior in weights; LoRA learns small low-rank adapters and QLoRA combines adapters with a quantized base. Alignment methods such as SFT, RLHF, and DPO target preferred behavior.

- Choose prompting for format, role, or rapidly changing behavior.
- Choose RAG for fresh, private, or citable knowledge.
- Choose fine-tuning for stable behavior at sufficient scale.
- Ground hallucination; do not expect prompting alone to remove it.

Behavioral claims need an external check

CHIVE investigates model behavior by editing prompts and measuring the resulting responses. In this course, a plausible explanation is a hypothesis to test. Schema checks, factual checks, and policy checks answer different questions.

- Predict the effect of one prompt edit.
- Score the response with a stated criterion.
- Distinguish measured behavior from a causal explanation.

Exam-ready summary

- An LLM is stochastic and stateless; the agent harness supplies control and state.
- Prompt structure and examples are the cheapest adaptation lever.
- Structured output and function calling make generations programmable.
- Prompting, RAG, and fine-tuning are complements, not substitutes.

Self-test

1. How do temperature and top-p affect an agent loop?
2. What competes for space in the context window?
3. Compare zero-shot, few-shot, chain-of-thought, self-consistency, and PAL.
4. Why is valid JSON insufficient as a security guarantee?
5. When should an engineer choose RAG rather than fine-tuning?
6. Why can a valid schema and a plausible explanation still accompany a wrong answer?

Assessed practice

Keep a factual task fixed and add one misleading cue. Predict whether the answer will change. Record both conditions and identify which check tests structure, evidence, or policy.

Acceptance check: The submission contains paired inputs and a testable prediction. It does not treat verbal reasoning as ground truth.

Lab: labs/07_agents_sdk_evaluation.ipynb

Reading and evidence

- P3 [Would this change your answer?](#). Anthropic/Fellows preprint, arXiv v1, 2026-08-17. CHIVE tests counterfactual prompt changes. Generated explanations remain hypotheses. Official post: August 21.
- S4 [Gemini structured outputs](#). Google documentation, accessed, 2026-09-09. Supported schemas constrain structure. Application validation must check meaning and policy.

03. Agent Anatomy and ReAct

ReAct operationalizes agency as a bounded Thought-Action-Observation cycle whose evidence comes from the runtime, not the model.

Learning objectives

- Engineer the four agent components
- Trace a ReAct episode
- Implement safe parsing and termination
- Recognize horizon and format failures

Core notes

The profile is an executable contract

The system prompt defines role, objective, tool menu, output protocol, constraints, stopping behavior, and escalation. It should read like a precise specification for a careful colleague, with examples where the protocol is easy to misunderstand.

- State success before listing implementation details.
- Give each tool a description and example call.
- Specify an exact output protocol and stopping condition.
- Add guardrails and human escalation rules.

Memory turns a stateless model into a stateful system

Short-term memory is the running context; long-term memory is an external store read and written through tools. The transcript is the immediate state of ReAct because each next decision depends on prior actions and observations.

- The application or provider must carry the bounded conversation state.
- Long-term memory is scalable but requires explicit retrieval.
- Only re-supplied information can influence the next model call.

Planning ranges from next-step choice to search

Decomposition creates subgoals, reasoning chooses actions, reflection critiques progress, and search explores alternatives. ReAct uses the simplest planner: decide only the next action from current evidence.

- Reactive planning adapts quickly to observations.
- One-step lookahead is efficient but myopic.
- Reflection and plan-and-execute address longer horizons.

The runtime owns the action boundary

The model proposes a tool and arguments. The runtime parses or receives the structured call, validates types and policy, authorizes it, executes the tool, and appends the real result as an observation. The model must never fabricate the observation.

- Use supported stop sequences as format controls in text protocols.
- Prefer native tool calling in production when available.
- Sandbox, scope permissions, and confirm consequential actions.

ReAct interleaves deliberation and grounding

Each iteration contains a Thought from the model, an Action from the model, and an Observation from the environment. Acting grounds later reasoning in evidence, while reasoning improves tool selection compared with an act-only policy.

- Thought chooses the next step.
- Action names and parameterizes a tool.
- Observation records the runtime result.
- Final Answer ends the episode.

A correct loop is bounded and observable

Termination must include a final-answer condition plus hard step, token, cost, or time budgets. The trace should be captured so evaluators can inspect tool choice, observations, errors, efficiency, and whether the final claim is grounded.

- Common failures: format drift, greedy parsing, fabricated observations.
- Long horizons compound errors and can make the agent lose the goal.
- No backtracking means a bad line may persist.
- Native schemas reduce format drift but not reasoning errors.

A complete tool cycle returns observed results

A native tool request is only part of the protocol. After validation, the runtime returns each result with the matching call identity and continues the model exchange. Preserve the provider's complete response content and cap the loop.

- Dispatch through the supplied tool registry.
- Keep model rounds and tool calls as separate budgets.
- Test empty responses and multiple calls.

Exam-ready summary

- The model proposes actions; the runtime validates and executes.
- ReAct alternates Thought, Action, and real Observation.
- Validate the complete protocol and enforce runtime budgets.
- Tracing reveals both outcome and trajectory failures.

Self-test

1. What belongs in a robust agent profile?
2. Who produces each part of Thought-Action-Observation?
3. What can a supported stop sequence prevent, and what must the runtime still check?
4. Compare text parsing with native function calling.
5. List the required termination conditions for a safe loop.
6. Which state must survive the model, tool, and model round-trip?

Assessed practice

Complete Lab 5. Demonstrate that the second model request contains the executed tool result and its matching ID. Replace the registry with a fake and prove that the replacement runs.

Acceptance check: The offline round-trip and replacement-registry tests pass. A missing result or exhausted budget yields an explicit stop reason.

Lab: labs/05_gemini_bounded_tools.ipynb

Reading and evidence

- S1 [Gemini function calling](#). Google documentation, accessed, 2026-09-09. Check model support and preserve complete model content when returning function responses.

04. Agentic Design Patterns

The best architecture keeps control in code when possible and hands decisions to models only where dynamic judgment adds value.

Learning objectives

- Recognize five workflow patterns
- Compare workflow and agentic control
- Compose patterns into a system
- Estimate reliability and cost trade-offs

Core notes

The augmented LLM is the reusable building block

Retrieval, tools, and memory turn a plain model call into an augmented LLM. Design patterns connect one or more augmented LLMs with control flow. The central design question is whether code or the model decides what happens next.

- Code-directed workflows are predictable and testable.
- Model-directed agents are flexible but variable.
- Add autonomy only when decisions cannot be enumerated reliably.

Chaining and routing encode known structure

Prompt chaining applies fixed ordered steps and can place validation gates between them. Routing classifies an input and selects a specialized handler. Both retain explicit control paths.

- Chaining fits outline-draft-polish style sequences.
- Gates can check schema, rules, or quality.
- Routing needs calibrated accuracy and a fallback lane.
- Hard or low-confidence cases can escalate.

Parallelization trades compute for speed or reliability

Sectioning divides independent work and aggregates results, reducing wall-clock time. Voting runs the same task several times and aggregates answers, increasing cost in exchange for robustness.

- Parallelize only independent subtasks.

- Aggregation is a distinct step that can fail.
- Reserve voting for decisions where redundancy is worth the cost.

Orchestrator-workers creates subtasks dynamically

An orchestrator decomposes the current input, dispatches variable subtasks to workers, and synthesizes their outputs. Unlike a chain, the exact work plan is not known in advance.

- Workers may be prompts, tools, or complete agents.
- The orchestrator must track state and integrate results.
- Dynamic decomposition increases flexibility and evaluation burden.

Evaluator-optimizer and reflection require grounded criteria

A generator produces an attempt, an evaluator checks it, and feedback drives revision. The loop improves reliably when criteria are externally verifiable, such as tests, schemas, calculations, rubrics, or reference answers.

- Vague 'is this good?' critics may rubber-stamp outputs.
- Tool-grounded critics can verify objective properties.
- Every refinement loop needs a stop and cost budget.

Patterns compose, so failure controls must compose too

A router can select an agent, an orchestrator can dispatch ReAct workers, and an evaluator can check the synthesis. Humans can approve, correct, or receive escalations at consequential boundaries.

- Anti-patterns: agent where workflow suffices, too many tools, unbounded loops.
- Each additional model call increases cost and failure opportunity.
- Use the simplest pattern whose control assumptions match the task.

An extra stage must justify its cost

A reviewer or worker adds another opportunity to help and another opportunity to fail. Use identical cases to compare a baseline with the proposed composition. Count review and coordination calls when evaluating the whole system.

- Use a fixed baseline before changing the architecture.
- Score outcomes and failures with the same rules.
- Account for overhead even when review leaves the answer unchanged.

Exam-ready summary

- Patterns connect augmented LLMs with different control structures.
- Chaining, routing, parallelization, orchestrator-workers, and evaluator-optimizer are workflows.
- Tool use, reflection, planning, and multi-agent systems hand more control to models.
- Ground checks and budgets at every loop boundary.

Self-test

1. What distinguishes a workflow from an agent?
2. Compare prompt chaining and orchestrator-workers.
3. Name the two forms of parallelization and their goals.
4. Why do evaluator-optimizer loops need verifiable criteria?
5. Give a valid composition of three patterns.
6. How can a reviewer increase cost without increasing correctness?

Assessed practice

Compare a single catalog agent with an objective evidence check. Specify when an LLM reviewer would add information that the objective check lacks.

Acceptance check: Include a case where review adds no value and one where it changes an incorrect proposal. Count all calls in the live variant.

Lab: labs/07_agents_sdk_evaluation.ipynb

Reading and evidence

- R1 [Patterns and problems in emerging multiagent systems](#). Anthropic research post, 2026-08-13. Controlled coordination experiments. Compare scope and budgets before interpreting the findings.

05. Tool Use, MCP, and Frameworks

Tools are typed prompts at a security boundary; MCP standardizes their integration, while frameworks organize control and state.

Learning objectives

- Design reliable tool schemas
- Explain the model-runtime-tool cycle
- Describe MCP primitives and transports
- Choose raw APIs or a framework

Core notes

A tool is a typed contract, not direct model access

A tool definition names an operation, explains when to use it, and defines typed parameters. The model emits a name and arguments; the runtime validates, authorizes, executes, and returns the result to context.

- The model never directly touches the external API.
- Schema validity must be followed by policy validation.
- Tool results are untrusted inputs to the next model step.

Tool descriptions are prompts

Models select and fill tools from their specifications, so names, descriptions, examples, parameter types, and error messages directly affect behavior. Good granularity avoids both excessive micro-calls and confusing mega-tools.

- Prefer minimal parameters and enums over unconstrained text.
- Return actionable errors rather than stack traces.
- Make retry-prone operations idempotent.
- Gate destructive or irreversible actions.

The runtime is the security boundary

Model-generated arguments must be treated as attacker-controlled input. Runtime controls include sanitization, least-privilege credentials, read-only defaults, sandboxing, rate limits, audit logs, and confirmation.

- Do not execute raw model strings with unsafe evaluators.
- Restrict filesystem and network scopes.
- Separate proposing an action from authorizing it.

MCP replaces bespoke N-by-M integration

Without a standard, every agent must be wired separately to every tool. The Model Context Protocol provides a shared JSON-RPC interface so hosts can connect to reusable servers, changing integration growth from N multiplied by M toward N plus M .

- Tools expose actions with possible side effects.
- Resources expose read-only context.
- Prompts expose reusable interaction templates.

MCP capabilities depend on the selected version

For the MCP 2025-11-25 baseline, servers expose tools, resources, and prompts. Clients can support sampling, roots, and elicitation. This edition discusses stdio and Streamable HTTP and requires implementations to enforce access controls.

- Sampling lets a server request host-model generation.
- Roots advertise filesystem context. Implementations enforce access.
- Elicitation asks the user for input during a task.
- Transport choice changes deployment and trust assumptions.

Frameworks package orchestration, not understanding

Frameworks provide loops or state graphs, persistence, streaming, tool integration, observability, memory, and human-in-the-loop hooks. Raw APIs suit simple or highly controlled agents; frameworks help when state and coordination become first-class.

- LangGraph represents nodes, edges, conditional control, and shared state.
- Crew-style systems emphasize role-based multi-agent work.
- Understand prompts and boundaries before adding framework abstraction.
- MCP standardizes tools; A2A standardizes agent-to-agent collaboration.

Tool interfaces and authority need separate checks

The labs use Gemini native calls and the OpenAI Agents SDK compatibility path. They expose local functions with explicit input limits. MCP descriptions and roots convey context, while the host and server implementations enforce access.

- Reject unknown names and extra arguments before execution.
- Bound input size and arithmetic magnitude.
- Check provider features rather than assuming parity.

Exam-ready summary

- A model requests a tool call; a runtime decides whether to execute it.
- Tool ergonomics strongly influence model reliability.
- MCP standardizes reusable tools, resources, and prompts.
- Frameworks are justified by stateful orchestration needs.

Self-test

1. Walk through the complete function-call cycle.
2. What makes a tool specification model-friendly?
3. Why must tool results be treated as untrusted?
4. Explain the N-by-M problem and MCP's answer.
5. List server-side and client-side MCP primitives.
6. Why does an advertised filesystem root still need implementation-level access controls?

Assessed practice

Complete Lab 5's dispatch policy. For the MCP 2025-11-25 baseline, explain why a roots response cannot substitute for filesystem access controls.

Acceptance check: Reject booleans in numeric fields, nonfinite values, and oversized input. Name the component that enforces each permission.

Lab: labs/05_gemini_bounded_tools.ipynb

Reading and evidence

- S1 [Gemini function calling](#). Google documentation, accessed, 2026-09-09. Check model support and preserve complete model content when returning function responses.

- S2 [OpenAI Agents SDK model integration](#). SDK documentation, accessed, 2026-09-09. The course uses local tools with a Gemini Chat Completions compatibility endpoint.
- S5 [MCP roots specification 2025-11-25](#). Versioned protocol specification, 2025-11-25. The course uses this historical protocol baseline. Implementations enforce access controls.

06. RAG and Agentic RAG

RAG grounds generation in controlled evidence; agentic RAG adds decisions about whether, what, and how to retrieve.

Learning objectives

- Explain the classic RAG pipeline
- Tune chunking and retrieval
- Evaluate retrieval and generation separately
- Compare vector, agentic, and graph retrieval

Core notes

Retrieval addresses the limits of parametric memory

Knowledge stored in weights can be outdated, unavailable for private corpora, unreliable on rare facts, and difficult to verify. RAG fetches evidence at query time and includes it in context so generation can be grounded and cited.

- Domain accuracy comes from a controlled corpus.
- Freshness comes from updating the index.
- Traceability comes from source metadata and citations.
- Privacy requires access control before retrieval reaches the model.

Classic RAG is an ingestion and query pipeline

Documents are loaded and normalized, split into chunks, embedded, and stored. At query time the question is embedded, relevant chunks are retrieved, the prompt is augmented, and the LLM generates an answer from that evidence.

- Loaders preserve metadata such as source, date, and permissions.
- Vector indexes support semantic nearest-neighbor search.
- The generated answer should cite the retrieved source.

Chunking sets the ceiling for retrieval

Chunks must fit budgets without cutting away the relationships needed to answer. Fixed-size windows are a practical default; hierarchical or sentence strategies fit structured or precise material. Overlap reduces boundary loss at additional index cost.

- Too-small chunks lose context.
- Too-large chunks dilute relevance.
- Answers spanning boundaries may require overlap or parent-child retrieval.
- Metadata captured during loading enables filters.

Naive top-k retrieval is only a baseline

Semantic similarity can miss exact terms and return redundant or weak evidence. Hybrid search combines vector and lexical retrieval; reranking improves precision; query transformation, expansion, multi-query, and HyDE improve recall.

- Top-k trades missed evidence against context dilution.
- Filter permissions before semantic ranking.
- Rerank a candidate set rather than the whole corpus.

Evaluate the retriever and generator separately

Retrieval metrics such as recall, precision, and MRR ask whether relevant evidence was fetched. Generation metrics ask whether the answer is relevant and faithful to that evidence. Diagnosing the two stages separately prevents prompt changes from hiding retrieval failures.

- Faithfulness directly measures unsupported claims.
- Citations make human verification possible.
- Production evaluation should include freshness and permission behavior.

Agentic and graph retrieval solve different hard cases

Agentic RAG lets the model decide whether to retrieve, rewrite or decompose queries, grade evidence, and retrieve again. GraphRAG extracts entities and relationships and retrieves connected subgraphs, which helps multi-hop questions and hidden connections.

- Self-RAG decides and critiques retrieval.
- Corrective RAG grades evidence and uses a fallback when weak.
- Vector RAG finds semantically relevant text.
- GraphRAG follows relationships at higher build cost.

Provenance and abstention make retrieval auditable

Carry source IDs and text through every retrieval step. Check whether the evidence supports the answer separately from whether a citation ID exists. A bounded repair attempt should end in an answer or an explicit abstention.

- Compare against static retrieval on the same questions.
- Keep conflicting evidence visible.
- Treat a rewrite as optional and measurable.

Exam-ready summary

- RAG provides fresh, private, and citable evidence.
- Chunking and retrieval quality determine the maximum answer quality.
- Measure retrieval and generation as separate systems.
- Agentic RAG controls retrieval; GraphRAG retrieves relationships.

Self-test

1. Describe the RAG pipeline from ingestion to answer.
2. How do chunk size and overlap affect retrieval?
3. Why can top-k retrieval dilute an answer?
4. Compare hybrid search, reranking, and query transformation.
5. What is faithfulness and why is it important?
6. What can a citation-membership test establish, and what remains untested?

Assessed practice

Complete Lab 6 and add unknown, conflicting, and irrelevant records. Compare zero repair with one repair using a fixed question set.

Acceptance check: Report retrieval hits, unsupported answers, abstentions, and attempts separately. A known citation ID alone does not count as grounded correctness.

Lab: labs/06_langgraph_corrective_rag.ipynb

Reading and evidence

- S4 [Gemini structured outputs](#). Google documentation, accessed, 2026-09-09. Supported schemas constrain structure. Application validation must check meaning and policy.

07. Memory, State, and Context Engineering

Memory is an engineered system that curates a bounded context, persists selected experience, and governs recall, expiry, and access.

Learning objectives

- Classify agent memory
- Manage short-term context
- Design deliberate long-term writes and reads
- Evaluate memory quality and governance

Core notes

Stateless models require explicit state engineering

An LLM knows only the current context. Long tasks and multiple sessions therefore need a system that decides what stays in the window, what moves to external storage, and what is retrieved later.

- The context is bounded, costly, and imperfectly used.
- Nothing persists across calls unless the application stores it.
- Memory architecture is a set of selection policies.

Short-term and long-term memory have different mechanics

Short-term memory is working context: instructions, tools, evidence, and recent trace. Long-term memory is external and can hold episodic events, semantic facts, and procedural knowledge across sessions.

- Episodic: past interactions and outcomes.
- Semantic: facts and stable preferences.
- Procedural: instructions or learned methods.
- Long-term storage is useful only if recall is selective.

Short-term memory must be actively compressed

Truncation drops old turns, summaries compress them, selective recall retrieves only relevant history, and a structured scratchpad keeps the plan and current state visible. These mechanisms can be combined.

- Rolling summaries preserve gist but may drift.
- Recent detail can remain verbatim.
- Conversation retrieval is RAG over history.
- A scratchpad reduces repeated reasoning and goal loss.

Long-term memory needs write and read policies

Store durable signal rather than every event. Writes can occur on explicit request, at task completion, through reflection, or continuously for changing state. Reads can use keys, recency, or semantic similarity.

- Prefer stable preferences, durable facts, outcomes, and lessons.
- Deduplicate and resolve contradictions.
- Attach timestamps, provenance, and expiry.
- Reflective consolidation turns noisy episodes into higher-level memories.

Context engineering treats tokens as scarce capacity

More context is not automatically better. Low-relevance history raises cost and latency and can create context rot. Long-context models provide more room but do not create cross-session persistence, selective recall, consent, or governance.

- Relevance beats volume.
- Place critical instructions and evidence deliberately.
- Virtual-context architectures page information between window and store.
- Retrieval-based architectures recall memories by relevance.

Memory is also a privacy and security system

Persistent data requires consent, minimization, retention limits, deletion, and access control. Memory can become stale, contradictory, poisoned, or unbounded, so it needs evaluation and maintenance.

- Recall: retrieve the right memory at the right time.
- Faithfulness: use the memory correctly.
- Forgetting: expire stale or irrelevant items.
- Security: prevent unauthorized access and malicious persistence.

Execution state and model context serve different purposes

A checkpoint stores workflow state for resumption. A context policy chooses what the next model call can see. Test these mechanisms separately. OpenAI's harness experiment motivates recording context settings as part of the evaluated system.

- Reopen a checkpoint before resuming approval.
- Keep side effects after the approval boundary.
- Compare context policies without changing the task set.

Exam-ready summary

- Memory is context plus external stores and policies.
- Short-term memory is curated; long-term memory is deliberate.
- Long context complements but does not replace memory.
- Recall, consolidation, expiry, consent, and access control are first-class.

Self-test

1. Why does an LLM need an application-level memory system?
2. Compare episodic, semantic, and procedural memory.
3. What are four ways to manage a full context window?
4. When should an agent write a long-term memory?
5. Why does a million-token context not eliminate memory architecture?
6. How can a workflow resume correctly while its next model call still lacks needed context?

Assessed practice

Complete Lab 8's checkpoint test. As an extension, compare a fixed recent-history window with a structured task summary on the same lookup tasks.

Acceptance check: Show the pending state before resume and one committed effect after a retry. Do not generalize a context-policy result across providers.

Lab: `labs/08_langgraph_approval_security.ipynb`

Reading and evidence

- S3 [LangGraph interrupts](#). LangChain documentation, accessed, 2026-09-09. Resume restarts the interrupted node. Put effects after approval and make replay safe.

- R2 [How enabling two settings tripled our scores on the ARC-AGI-3 benchmark](#). OpenAI research post, 2026-07-29. A specific harness experiment. Its result does not estimate the effect of context changes in Gemini.

08. Multi-Agent Systems

Multiple agents add specialization, modularity, parallelism, and context division only when coordination costs are explicitly designed.

Learning objectives

- Decide when multiple agents help
- Design roles, communication, and coordination
- Compare centralized, decentralized, and hierarchical topologies
- Bound multi-agent cost and termination

Core notes

More agents are useful only for separable work

A focused role, prompt, tool set, and context can outperform one overloaded agent. Multiple agents also allow modular testing, parallel tasks, and divided context. These gains disappear when work is tightly coupled or one well-equipped agent already succeeds.

- Specialization narrows behavior and tool choice.
- Modularity makes agents independently replaceable.
- Parallelism reduces time only for independent subtasks.
- Context division keeps each working set relevant.

Coordination is the price of specialization

Each agent adds model calls and communication. Errors can propagate through handoffs, disagreement needs resolution, and someone must decide when the team is done. Multi-agent designs are harder to trace and secure.

- Cost grows with agents, turns, and coordination rounds.
- Messages consume context and can carry errors or injections.
- Termination needs shared budgets and a clear owner.
- Start with a single agent and add roles for measured reasons.

Design collaboration along four dimensions

Roles define responsibility; communication defines information exchange; coordination selects the next actor and completion rule; evolution determines whether agents adapt through feedback or reflection.

- Give roles distinct objectives and tool scopes.
- Choose shared state or explicit messages deliberately.
- Define handoff contracts and synthesis ownership.
- Keep adaptation separate from uncontrolled drift.

A runtime supplies identity, lifecycle, and delivery

The application sits on a runtime that creates and retires agent instances, uniquely addresses them, and routes messages. Standalone runtimes are simpler; distributed runtimes add process, machine, language, isolation, and operations concerns.

- Use one process for development and simple applications.
- Distribute only for scale, isolation, or polyglot requirements.
- Persist state intentionally across instance lifecycles.

Communication can be shared-state or message-based

A blackboard lets agents read and update common state. Direct messaging targets a specific agent. Publish-subscribe decouples publishers from subscribers through topics. Topic scope is also a security boundary.

- Shared state simplifies synthesis but can create coupling.
- Direct messaging makes ownership explicit.
- Pub-sub supports fan-out and looser coupling.
- Partition topics by user, session, or tenant to prevent leakage.

Topology determines control and failure shape

A centralized supervisor decomposes, routes, and synthesizes with clear accountability but becomes a bottleneck. Peer-to-peer systems remove the single boss but are harder to control. Hierarchies compose teams at scale while multiplying coordination layers.

- Supervisors behave like LLM routers over shared state.
- Peers need negotiation, consensus, and termination rules.
- Hierarchies need bounded delegation at every level.
- Measure handoffs, agent-specific failures, and total call budget.

Shared evidence matters more than agreement

Anthropic's multi-agent experiments include information-sharing failures and conflicting objectives. For a classroom comparison, keep task scope and budget explicit. Agreement between agents does not establish independent evidence.

- Inspect which facts each role receives.
- Test a misleading cue shared across agents.
- Give one component responsibility for synthesis and termination.

Exam-ready summary

- Use multiple agents for genuine specialization or parallelism.
- A runtime provides identity, lifecycle, and messaging.
- Communication and topology determine security and debugging complexity.
- Every team needs an owner of synthesis, budget, and completion.

Self-test

1. List four benefits and four costs of multi-agent systems.
2. What are the four dimensions of collaboration?
3. Compare shared state, direct messaging, and publish-subscribe.
4. When is a distributed runtime justified?
5. Compare centralized, decentralized, and hierarchical topologies.
6. Why can a team agree on an incorrect answer even when its members sample separately?

Assessed practice

Complete Lab 7. Compare a single agent, an objective reviewer, and an optional model reviewer. Add one case where every role sees the same misleading cue.

Acceptance check: Report extra requests and shared errors. Explain any scope or information advantage before comparing outcomes.

Lab: labs/07_agents_sdk_evaluation.ipynb

Reading and evidence

- R1 [Patterns and problems in emerging multiagent systems](#). Anthropic research post, 2026-08-13. Controlled coordination experiments. Compare scope and budgets before interpreting the findings.
- S2 [OpenAI Agents SDK model integration](#). SDK documentation, accessed, 2026-09-09. The course uses local tools with a Gemini Chat Completions compatibility endpoint.

09. Multi-Agent Interoperability

A2A treats agents as discoverable services with long-running task lifecycles, while MCP connects each agent to tools and data.

Learning objectives

- Explain the agent interoperability problem
- Describe Agent Cards and A2A task objects
- Separate MCP and A2A responsibilities
- Threat-model cross-agent delegation

Core notes

Agent ecosystems recreate the integration problem one layer up

Real agents use different frameworks, vendors, organizations, endpoints, and security domains. Bespoke pairwise integrations do not scale. An interoperable agent must publish capabilities and accept work without exposing its internal reasoning implementation.

- An agent resembles a service with discovery, endpoint, and authentication.
- Unlike a simple API, it reasons and may run for minutes or days.
- The protocol must cover status, streaming, and artifacts.

A2A 1.0.0 separates operations from bindings

A2A 1.0.0 defines a task and message model with JSON-RPC, gRPC, and HTTP/REST bindings. This course traces JSON-RPC over HTTP. Select a binding and version explicitly when implementing discovery, streaming, and authentication.

- This course uses the JSON-RPC binding over HTTP.
- Other defined bindings include gRPC and HTTP/REST.
- The JSON-RPC streaming path uses server-sent events.
- The application authenticates and authorizes each caller.

Agent Cards make capabilities discoverable

An Agent Card is a machine-readable manifest containing identity, provider, version, skills, endpoint, and authentication information. Clients can fetch a well-known URL, use a registry, or follow a referral.

- Discovery avoids hard-coding every remote capability.
- Cards should be authenticated or signed before trust.
- Capabilities should be described at agent-level granularity.

Tasks organize long-running collaboration

A task has a lifecycle such as submitted, working, waiting for input, completed, failed, or canceled. Messages contain parts such as text or files; artifacts are the produced deliverables. Streaming exposes progress without pretending the work is one synchronous call.

- Messages carry conversation turns.
- Parts carry multimodal content.
- Artifacts carry completed outputs.
- Lifecycle state enables pause, input, retry, and completion.

MCP and A2A form two different layers

MCP connects an agent to schema-defined tools and data sources. A2A connects one autonomous, reasoning agent to another through a task lifecycle. A remote agent may itself use MCP servers to complete delegated work.

- Use MCP for vertical integration with tools and data.
- Use A2A for horizontal collaboration across agent boundaries.
- Do not model a long-running autonomous service as a micro-tool.

Delegation crosses trust and governance boundaries

Every cross-agent edge raises questions about identity, authorization, prompt injection, over-delegation, data retention, compliance, cost, and audit. Remote outputs must be handled as untrusted content even after authentication.

- Authenticate the agent and verify its card.
- Scope delegated authority by task, spend, action, and time.
- Confirm high-stakes actions with a human.
- Log context leaving the organization and artifacts returning.

Delegation needs a versioned contract

This chapter uses A2A 1.0.0 and traces its JSON-RPC binding. The specification also defines gRPC and HTTP/REST bindings. Identity, permission, and the validity of a returned artifact remain separate checks.

- Record the protocol version and selected binding.
- Authenticate the caller before assigning authority.
- Treat returned content as untrusted evidence.

Exam-ready summary

- A2A standardizes discovery and long-running work between agents.
- Agent Cards advertise identity, skills, endpoint, and authentication.
- MCP serves tools and data; A2A serves agent collaboration.
- Cross-agent trust requires authentication, least privilege, bounds, and audit.

Self-test

1. Why is an agent not equivalent to a simple REST endpoint?
2. What information belongs in an Agent Card?
3. Define task, message, part, and artifact.
4. How does A2A support long-running work?
5. Compare MCP and A2A with a concrete example.
6. Which responsibilities remain in the application after it adopts an interoperability protocol?

Assessed practice

Trace a delegated catalog task from discovery to completion. Identify the caller, authorization decision, timeout owner, and artifact validator. No remote service deployment is required.

Acceptance check: Use A2A 1.0.0 terminology and distinguish the chosen binding from the abstract task model. Explain what happens after a failed or canceled task.

Lab: labs/08_langgraph_approval_security.ipynb

Reading and evidence

- S6 [A2A specification 1.0.0](#). Versioned protocol specification, accessed, 2026-09-09. Separates the data model from JSON-RPC, gRPC, and HTTP/REST bindings. The course traces JSON-RPC.

10. Advanced Reasoning and Planning

Reasoning strategies trade inference compute for exploration, correction, or structure; the correct strategy depends on task difficulty and verifiability.

Learning objectives

- Compare linear, sampled, searched, and reflective reasoning
- Choose a planning strategy
- Explain test-time compute scaling
- Recognize overthinking and faithfulness limits

Core notes

A single chain is useful but commits early

Chain-of-thought provides one linear path. It can make intermediate work inspectable, but a greedy chain has no exploration or recovery and its verbalized explanation is not guaranteed to be a faithful record of internal computation.

- Compare a direct baseline with explicit reasoning on the task.
- Verify outputs externally where possible.
- Do not treat stated reasoning as ground truth.

Self-consistency widens exploration by sampling

Self-consistency samples multiple reasoning paths and aggregates their answers. Separately sampled outputs may share systematic errors. Measure whether voting improves the target task under a declared budget.

- Cost grows roughly with the number of samples.
- Voting requires an answer that can be aggregated.
- Measure shared errors rather than assuming independence.

Tree- and Graph-of-Thoughts add explicit search

Tree-of-Thoughts generates candidate partial states, evaluates them, and expands promising branches using a search strategy, enabling lookahead and backtracking. Graph-of-Thoughts also merges and refines ideas rather than keeping a strict tree.

- Search needs a useful evaluator.
- Branching can grow exponentially.
- Graphs support aggregation and refinement loops.
- Use search when alternatives and dead ends matter.

Reflection improves one path over time

Reflexion converts evaluation into a verbal lesson stored for another attempt. Self-Refine repeats critique and revision. Reflection deepens one trajectory, whereas search widens across trajectories.

- A test or grounded critic makes reflection reliable.
- Vague self-critique can reinforce errors.
- Revision loops need quality and cost stops.

Planning changes when and how decisions are made

ReAct decides step by step and suits short uncertain tasks. Plan-and-execute decomposes first and can reduce repeated planning on long tasks. Re-planning repairs a plan after new evidence. DAG planning runs independent steps in parallel.

- Reactive plans adapt but can be myopic.
- Up-front plans add global structure but can become stale.
- Parallel plans lower latency only when dependencies permit.
- Every planner needs execution feedback.

Reasoning models internalize deliberate inference

RL-trained reasoning models generate extended internal deliberation and spend test-time compute according to task difficulty. Strong models can serve as planners or orchestrators while cheaper models handle routine extraction, routing, and tool calls.

- More thinking raises latency and cost.
- Easy problems can suffer from overthinking.
- Beyond a point, additional compute may reduce accuracy.
- Escalate selectively instead of using maximum reasoning everywhere.

Reasoning strategies require controlled comparisons

Compare a direct baseline with sampling or review under a declared resource budget. Separate samples can share systematic errors. Counterfactual prompt edits test a behavioral claim without establishing a complete account of internal computation.

- Choose a scorer before inspecting answers.
- Keep held-out cases out of prompt development.
- Report when extra reasoning fails to help.

Exam-ready summary

- Self-consistency samples; search branches; reflection revises.
- ReAct, plan-and-execute, re-planning, and DAG execution solve different control problems.
- Reasoning compute should be allocated by difficulty.
- Verbal reasoning is a scaffold, not proof of correctness.

Self-test

1. Compare chain-of-thought and self-consistency.
2. What are generate, evaluate, and search in Tree-of-Thoughts?
3. How does Graph-of-Thoughts extend a tree?
4. Compare reflection with search.
5. When should an agent use ReAct, plan-and-execute, or parallel planning?
6. What evidence would justify spending more inference on a reasoning strategy?

Assessed practice

Use Lab 7's paired inputs to test a prediction about a misleading cue. Propose an equal-budget comparison with voting and identify a shared-error failure case.

Acceptance check: Keep predictions, observations, and interpretations separate. State the limits of a small experiment.

Lab: labs/07_agents_sdk_evaluation.ipynb

Reading and evidence

- P3 [Would this change your answer?](#). Anthropic/Fellows preprint, arXiv v1, 2026-08-17. CHIVE tests counterfactual prompt changes. Generated explanations remain hypotheses. Official post: August 21.
- R1 [Patterns and problems in emerging multiagent systems](#). Anthropic research post, 2026-08-13. Controlled coordination experiments. Compare scope and budgets before interpreting the findings.

11. Evaluation and Observability

Agent quality requires systematic measurement of both final outcomes and the trajectories that produced them, supported by complete traces.

Learning objectives

- Design outcome and trajectory metrics
- Read benchmarks critically
- Use LLM judges with safeguards
- Build offline, online, and CI evaluation

Core notes

Agent evaluation is difficult for structural reasons

Runs are non-deterministic, outputs are open-ended, tasks are multi-step, and correctness, cost, latency, and safety can conflict. Models, tools, and prompts also change, so anecdotal testing cannot distinguish progress from regression.

- Use repeated runs where variance matters.
- Define several metrics rather than one headline score.
- Version the complete system under evaluation.

Outcome and trajectory answer different questions

Outcome evaluation asks whether the final answer or task result is correct. Trajectory evaluation asks whether the agent used appropriate tools, safe actions, efficient steps, and acceptable time and cost.

- Exact match suits unambiguous answers.
- Execution tests suit code and queries.
- Trajectory metrics localize the failing step.
- Safety and policy adherence belong in trajectory evaluation.

Objective scorers are preferable where available

Execution-based and exact scoring are easier to reproduce than reference similarity or an LLM judge. For RAG, retrieval and generation need separate metrics; for memory and multi-agent systems, evaluate recall, handoffs, agent-specific errors, and total cost.

- RAG: retrieval recall/precision/MRR plus faithfulness and relevance.
- Memory: right item, right time, correct use, appropriate forgetting.
- Multi-agent: handoff quality, ownership, synthesis, and call budget.

Benchmarks measure systems, not just base models

GAIA tests multi-step assistant behavior, SWE-bench Verified tests repository issue resolution through execution, and tool-dialogue benchmarks test policy adherence. Scores depend on the model, prompt, tools, interface, scaffold, and evaluation harness.

- Watch for training-data contamination.
- Weak tests may accept incorrect solutions.
- Benchmark tasks may not match production distribution.
- Ask whose system produced the number.

LLM-as-judge is flexible but biased

A judge model can score open-ended work at scale, but may favor position, verbosity, style, or its own outputs. Explicit rubrics, pairwise comparisons, swapped order, calibration against humans, and multiple judges reduce these risks.

- Prefer checkable criteria over a vague quality score.
- Blind the judge to irrelevant identity information.
- Audit disagreements and drift.

Tracing turns evaluation into engineering

A run trace is a tree of spans for model calls, tools, retrieval, and agents. Each span records inputs, outputs, timing, tokens, cost, errors, and relevant metadata. Offline eval protects releases; online eval monitors live behavior; CI gates block regressions.

- Build representative tasks with expected outcomes.
- Include hard, edge, adversarial, and safety cases.
- Run the same set on prompt, model, tool, or dependency changes.
- Alert on quality, latency, cost, and safety drift.

A run manifest makes comparisons reviewable

Record the complete setup and every attempted run. A completed answer can be wrong, and an infrastructure failure is a different outcome. Fixed cases and explicit denominators make a comparison inspectable.

- Store model ID, prompt hash, and dependency versions.
- Record attempted and completed runs separately.
- Keep outcome scoring separate from resource use.

Exam-ready summary

- Evaluation is continuous system steering, not a final phase.
- Measure both task success and the path taken.
- Treat benchmark scores as scaffold-dependent evidence.
- Trace every model, tool, retrieval, and agent step.

Self-test

1. Why is agent evaluation harder than single-answer evaluation?
2. Give four outcome metrics and four trajectory metrics.
3. What makes execution-based evaluation strong?
4. How can benchmark contamination and weak tests mislead?
5. What biases affect LLM-as-judge?
6. How should an evaluation distinguish a wrong answer from an API failure?

Assessed practice

Run the offline evaluation command, inspect its JSONL records, and test the failure path. For live inference, predeclare a model and budget before using the held-out split.

Acceptance check: A fresh rerun produces a manifest and one record per attempted case. Interrupted or failed requests do not disappear from the denominator.

Lab: labs/07_agents_sdk_evaluation.ipynb

Reading and evidence

- P3 [Would this change your answer?](#). Anthropic/Fellows preprint, arXiv v1, 2026-08-17. CHIVE tests counterfactual prompt changes. Generated explanations remain hypotheses. Official post: August 21.
- R2 [How enabling two settings tripled our scores on the ARC-AGI-3 benchmark](#). OpenAI research post, 2026-07-29. A specific harness experiment. Its result does not estimate the effect of context changes in Gemini.

12. Safety and Security

Action amplifies model errors and attacks, so safety must be enforced through defense-in-depth at the model-runtime boundary and throughout the lifecycle.

Learning objectives

- Separate adversarial security from responsible-AI harms
- Explain prompt injection and excessive agency
- Apply defense-in-depth
- Design runtime guardrails and safety evaluation

Core notes

Tools change the threat model

A model that only produces text has limited direct effect; an agent can send messages, execute code, modify files, or make transactions. Autonomy may remove per-step review, and untrusted web pages, documents, emails, and tool results enter the same context as instructions.

- The model proposes; the runtime authorizes.
- Real-world effects require stronger controls than content generation.
- Consequential actions need explicit policy and confirmation.

Security and responsible AI overlap but are distinct

Security addresses adversaries exploiting the agent through prompt injection, tool misuse, data exfiltration, memory poisoning, and excessive agency. Responsible AI addresses harmful, biased, toxic, false, or otherwise unsafe outputs.

- Least privilege and sandboxing reduce adversarial impact.
- Alignment and content filters reduce harmful output.
- A jailbreak can connect the two categories.

Prompt injection exploits the shared instruction-data channel

Direct injection is supplied by a user. Indirect injection is hidden in content the agent retrieves. Because instructions and data enter the same context, the model may follow malicious text as if it were authorized guidance.

- Indirect attacks require only control of content the agent reads.
- Tool results must never automatically become trusted instructions.
- There is no complete prompt-only fix.

Excessive agency amplifies every successful exploit

An injected instruction inherits the agent's available tools and credentials. Too many tools, broad scopes, persistent secrets, and unreviewed autonomy increase maximum harm.

- Grant minimum tools and scopes.
- Default to read-only and short-lived credentials.
- Bound actions, time, spend, and network access.
- Require confirmation for destructive or irreversible operations.

Defense-in-depth must be structural

Input validation, untrusted-content quarantine, tool-call policy, sandboxing, output filtering, monitoring, and human approval provide independent layers. A guardrail prompt or guardrail model can be fooled and therefore cannot be the sole defense.

- Validate before the model and again before tool execution.
- Separate trusted control data from retrieved content.
- Sanitize tool output before execution or rendering.
- Log and detect abnormal trajectories.

Responsible output needs training-time and runtime controls

Bias and toxicity can originate in web-scale training data; alignment methods such as RLHF and DPO shape preferred behavior. Runtime classifiers and guardrails screen inputs and outputs. Hallucination requires grounding, citations, abstention, and verification.

- Measure toxicity and bias rather than assuming alignment solved them.
- Use RAG and tools for factual grounding.
- Apply safety controls across MCP servers and A2A delegation.
- Include injection and disallowed-action cases in evaluation.

Security tests distinguish proposals from effects

A malicious proposal and a successful unauthorized effect are different outcomes. GPT-Red motivates evaluating held-out attacks, while this course tests a small application policy. Passing these fixtures does not establish general prompt-injection robustness.

- Bind approval to the exact proposed action.
- Measure false rejection on benign requests.
- Retain held-out adversarial cases.

Exam-ready summary

- Action moves safety enforcement into code and infrastructure.
- Prompt injection cannot be solved by a stronger prompt alone.
- Least privilege limits blast radius.
- Security, responsible AI, evaluation, and observability must work together.

Self-test

1. Why does agent action raise the stakes compared with a chatbot?
2. Differentiate security risks and responsible-AI harms.
3. Compare direct and indirect prompt injection.
4. What is excessive agency and how is it mitigated?
5. Design a defense-in-depth stack for a tool-using agent.
6. Why should a security report score model proposals and executed effects separately?

Assessed practice

Complete Lab 8. Submit one valid request, one injection-shaped proposal, and one replay with changed arguments. Identify the trusted reviewer channel.

Acceptance check: Rejected actions create no ledger entry. A retry with the same ID cannot duplicate the effect, and changed payload reuse fails.

Lab: labs/08_langgraph_approval_security.ipynb

Reading and evidence

- P1 [GPT-Red: Automated Red Teaming via Self-Play at Scale](#). OpenAI technical paper, 2026-07-15. Self-play and held-out adversarial evaluation. Classroom policy checks do not reproduce this training.
- S3 [LangGraph interrupts](#). LangChain documentation, accessed, 2026-09-09. Resume restarts the interrupted node. Put effects after approval and make replay safe.

13. Production, Economics, and the Frontier

Production success is determined by the versioned, observable, economical, and secure system around the model rather than a one-off demo.

Learning objectives

- Close the prototype-to-production gap
- Engineer cost, latency, and reliability
- Explain AgentOps and safe rollout
- Connect current successes to open problems

Core notes

A product must work repeatedly under constraints

A prototype needs one successful demonstration; a production service must work safely, reliably, economically, and at scale. Prompts, tools, models, retrieval, memory, policies, and evals become versioned system components.

- Reliability includes retries, timeouts, fallbacks, and idempotency.
- Compliance and security join functional correctness.
- The runtime, tools, and operating process need their own tests.

Cost and latency compound across agent steps

Agent loops make multiple model and tool calls, so per-token and per-call costs accumulate. Routing sends ordinary work to cheaper models and hard work to stronger reasoning models. Caching, smaller models, fine-tuning, and bounded loops reduce spend.

- Cache embeddings, retrievals, results, and prompt prefixes.
- Parallelize independent steps.
- Stream progress to reduce perceived latency.
- Set token, step, time, and spend budgets.

AgentOps combines service operations with LLM-specific controls

Production agents need traces, dashboards, alerts, prompt and model versioning, evaluation gates, guardrails, and human checkpoints in addition to normal service reliability patterns.

- Trace model, tool, retrieval, memory, and agent spans.
- Monitor quality, cost, latency, error, and safety metrics.
- Use circuit breakers and fallbacks for unstable dependencies.
- Preserve reproducibility across silent model or tool changes.

Rollout should be reversible

Agents can ship as services, batch workers, or embedded assistants. Canary releases, feature flags, gradual traffic, evaluation gates, and rapid rollback reduce the impact of behavioral regressions.

- Test on representative and adversarial tasks before deployment.
- Require approval for consequential actions.
- Keep a human-owned escalation path.
- Audit actions after deployment.

Execution-grounded domains reveal the clearest wins

Coding agents can inspect repositories, edit files, and run tests; their Agent-Computer Interface exposes compact actions and concise feedback. Computer-use agents generalize to GUIs through screenshots or accessibility trees but remain more brittle.

- Interface design can matter as much as the base model.
- Execution tests make coding outcomes verifiable.
- Human review remains essential.
- High benchmark scores do not guarantee open-world reliability.

The frontier and the open problems advance together

Reasoning models, longer autonomous horizons, computer use, multimodality, MCP, and A2A expand capability. Reliability over long horizons, evaluation leakage, prompt injection, cost, accountability, labor impact, and misuse remain unresolved constraints.

- Clear ownership is required for deployed agents.
- Audit trails document what happened and why.
- Economics differ from fixed-cost traditional software.
- Launch only the least autonomous system that passes eval and safety gates.

A maintained system needs evidence and an owner

The scientific-computing field report motivates reference tests and stewardship. OpenAI's observational research post also warns that activity metrics need careful interpretation. The capstone therefore assesses validated outcomes and reproducibility.

- Test recovery and unexpected inputs.
- Name the maintainer and escalation path.
- Separate activity counts from demonstrated task success.

Exam-ready summary

- Production is disciplined engineering around the model.
- Routing, caching, bounds, and parallelism control cost and latency.
- AgentOps versions, evaluates, traces, guards, and rolls back the whole system.
- Frontier capability does not remove reliability, security, or accountability limits.

Self-test

1. What distinguishes an agent demo from a production product?
2. Give four methods for reducing cost and four for reducing latency.
3. What artifacts must be versioned for reproducibility?
4. What does AgentOps add to ordinary service operations?
5. Why are coding agents comparatively successful?
6. What evidence and ownership must accompany a successful agent demonstration?

Assessed practice

Submit the capstone bundle: baseline, bounded agent, held-out evaluation, negative tests, and an operating note. Include a reproducible command and a maintenance owner.

Acceptance check: The reviewer can rerun the bundle in a fresh environment and trace each conclusion to saved evidence. Follow teaching/CAPSTONE.md for the rubric.

Lab: labs/08_langgraph_approval_security.ipynb

Reading and evidence

- P2 [Scientific computing in the age of agentic AI](#). OpenAI-affiliated exploratory field report, 2026-07-28. Case studies motivate verification and maintenance ownership. They do not estimate a universal productivity effect.
- R3 [Research acceleration: The view inside OpenAI](#). OpenAI observational research post, 2026-09-06. Activity metrics do not by themselves identify causal gains in research progress.

Exam Preparation Guide

Six comparisons to master

Compare	Essential distinction
Chatbot vs workflow vs agent	Reply-only vs code-directed control vs model-directed control
Prompting vs RAG vs fine-tuning	Behavior in context vs external knowledge vs weight adaptation
ReAct vs plan-and-execute	Next-step reactive choice vs global plan followed by execution
Workflow vs multi-agent	Explicit control paths vs coordinated autonomous roles
MCP vs A2A	Agent-to-tool/data integration vs agent-to-agent collaboration
Outcome vs trajectory evaluation	Whether the result is correct vs how it was produced

Four diagrams to reproduce from memory

1. The perceive-reason-act loop with the runtime at the action boundary.
2. The classic RAG ingestion and query pipeline.
3. The two-layer MCP + A2A architecture.
4. The production loop: build -> eval gate -> guardrails -> deploy -> trace and monitor.

High-value design rules

- Choose the least autonomy that solves the problem.
- The model proposes; the runtime validates, authorizes, and executes.
- Relevance beats context volume.
- Evaluate retrieval and generation separately.
- Add agents only for measured specialization, modularity, parallelism, or context division.
- Prefer objective and execution-based evaluation.
- You cannot prompt your way out of prompt injection.

- Version prompts, tools, models, policies, data, and evals.

Practice method

For a coding assessment, submit the input cases, expected results, versioned setup, and observed failures. Compare a simpler baseline before adding autonomy. Distinguish a malformed response, a wrong answer, an API failure, and a denied action. The capstone rubric appears in `teaching/CAPSTONE.md`.

For every architecture question, answer in four passes: define the components, trace control flow, identify failure modes, then state evaluation and safety controls. This mirrors how the source course develops each topic and prevents answers that describe capability without engineering discipline.

Glossary

A2A. A protocol for discovery and long-running collaboration between autonomous agents.

Action. A model-proposed tool call that the runtime validates, authorizes, and executes.

Agent. A system that uses an LLM as a reasoning core inside a loop to pursue a goal through actions.

Agent Card. A machine-readable manifest describing an agent's identity, skills, endpoint, and authentication.

AgentOps. Operational practices for versioning, evaluating, tracing, guarding, deploying, and monitoring agents.

Augmented LLM. An LLM combined with retrieval, tools, and memory.

Chain-of-thought. A linear sequence of intermediate reasoning steps used to scaffold a multi-step answer.

Context engineering. Selecting and arranging instructions, tools, evidence, memory, and history within a token budget.

Evaluator-optimizer. A workflow in which an evaluator gives criteria-based feedback to a generator for revision.

GraphRAG. Retrieval over entities and relationships in a knowledge graph.

Long-term memory. External persistent storage for selected episodic, semantic, or procedural information.

MCP. A protocol that standardizes connections between LLM hosts and tools, resources, and prompts.

Observation. The real result returned by the runtime or environment after an action.

Orchestrator-workers. A pattern in which a model dynamically decomposes work, delegates subtasks, and synthesizes results.

Profile. The role, objective, constraints, tools, and output protocol that define an agent's behavior.

RAG. Retrieval-augmented generation: fetching external evidence at query time and adding it to model context.

ReAct. A loop that interleaves Thought, Action, and Observation.

Reflection. Critiquing an attempt and using feedback to revise or guide a later attempt.

Self-consistency. Sampling multiple reasoning chains and aggregating their answers, usually by majority vote.

Short-term memory. The bounded current context containing instructions, evidence, and recent task history.

Tool. A typed operation available to the model through a controlled runtime.

Trajectory evaluation. Evaluation of tool choices, steps, cost, latency, and safety during an agent run.

Workflow. LLM calls and tools connected by code-defined control paths.